

# Joins and Data Modification in SQL

## IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management  
Oklahoma State University

Fall 2026

*Slides courtesy Dr. Manjunath Kamath, IEM, OSU.*

- 1 Retrieving Data from More Than One Table
- 2 Modifying Data with SQL

# Retrieving Data from More Than One Table

## Two Kinds of Join Syntax

- SQL supports two syntaxes for an inner join between two tables, the traditional syntax used through SQL-89 and SQL-92, and SQL-92 join syntax.
- Traditional join syntax combines the relational algebra product and restrict operations. Listing the tables after FROM requests the product, and the join condition in the WHERE clause's predicate requests the restrict that identifies which rows belong to the joined tables.

```
SELECT columns  
FROM table1, table2  
WHERE table1.primary_key = table2.foreign_key;
```

- Most DBMSs today use a faster method than an actual product followed by a restrict, though a join remains one of the slowest common SQL operations, since a relational database by its very nature requires a lot of them.

## A Traditional Join Example

```
SELECT first_name, last_name, sale_id, sale_date  
FROM customer, sale  
WHERE customer.customer_num = sale.customer_num  
AND contact_phone = '555-666-7777';
```

- This query shows orders placed by a customer with a given phone number. Phone number lives in the customer table, and purchase information lives in the sale table, related by customer number, which is the primary key in customer and a foreign key in sale.
- The table name is needed as a qualifier because `customer_num` appears in both tables. The query optimizer decides whether to join first and then filter by phone number, or filter by phone number first and then join.

## SQL-92 Join Syntax

- SQL-92 join syntax is simpler and more flexible than the traditional syntax, though it is not implemented by every DBMS.
- A natural join joins over every column with the same name in both tables. Using JOIN alone defaults to a natural equi-join, and the DBMS automatically finds every matching column name.

```
SELECT first_name, last_name, sale_id, sale_date  
FROM customer NATURAL JOIN sale  
WHERE contact_phone = '555-666-7777';
```

## Joining Over Selected or Differently Named Columns

- A USING clause joins over specific columns, useful when more than just the matching columns share a name.

```
SELECT first_name, last_name, sale_id, sale_date  
FROM customer JOIN sale USING (customer_num)  
WHERE contact_phone = '555-666-7777';
```

- An ON clause joins over columns with different names, needed whenever a foreign key's name does not match its matching primary key's name.

```
SELECT first_name, last_name, sale_id, sale_date  
FROM customer JOIN sale  
    ON customer.customer_num = sale.buyer_num  
WHERE contact_phone = '555-666-7777';
```

## Joining on Concatenated Keys

```
accountant (acct_first_name, acct_last_name, date_hired,  
            office_ext)
```

```
customer (customer_num, first_name, last_name, street,  
          city, state_province, zip_postcode, contact_phone)
```

```
project (tax_year, customer_num, acct_first_name,  
         acct_last_name)
```

```
form (tax_year, customer_num, form_id, is_complete)
```

- Consider a query asking which accountant worked on which form during which year. The project and form tables must be joined using a concatenated key, of `tax_year` and `customer_num` together.

```
SELECT acct_first_name, acct_last_name, form.tax_year,  
       form.form_ID  
FROM project, form  
WHERE project.tax_year = form.tax_year  
      AND project.customer_num = form.customer_num;
```

## Concatenated Keys in SQL-92 Syntax

- If the matching columns have the same names in both tables and are the only matching columns, a plain JOIN is enough.

```
SELECT acct_first_name, acct_last_name, form.tax_year,  
       form.form_ID  
FROM project JOIN form;
```

- If the columns share names but are not the only matching columns, list them explicitly in a USING clause.

```
SELECT acct_first_name, acct_last_name, form.tax_year,  
       form.form_ID  
FROM project JOIN form USING (tax_year, form_ID);
```

## Joining More Than Two Tables

- Consider a query asking for the names of everyone who purchased a volume with a given ISBN. The query must join volume to sale to find the relevant sales, then join that result to customer to get the names.

```
SELECT first_name, last_name  
FROM customer, sale, volume  
WHERE volume.sale_id = sale.sale_id  
      AND sale.customer_num = customer.customer_num  
      AND isbn = '978-1-11111-136-1';
```

- A join works with two tables at a time, so joining three tables needs two joins, and so on.

## Joining a Table to Itself

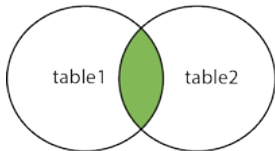
- A same-table join can answer questions such as which customers purchased two specific books.

```
SELECT first_name, last_name
FROM volume T1, volume T2, sale, customer
WHERE T1.isbn = '978-1-11111-146-1'
      AND T2.isbn = '978-1-11111-122-1'
      AND T1.sale_id = T2.sale_id
      AND T1.sale_id = sale.sale_id
      AND sale.customer_num = customer.customer_num;
```

- T1 and T2 are aliases for two separate references to the same volume table, letting each row selection criterion apply to a distinct copy of it.

# Inner Joins and Outer Joins

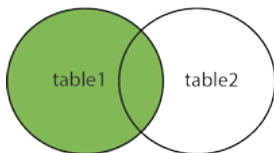
INNER JOIN



- Every join so far has been an inner join. A row appears in the result only when it has a matching row in both tables.
- That has a consequence which is easy to miss. Rows with no match simply disappear from the result, and nothing warns you.
- In the warehouse database used in the tutorials there are ten vendors but only five shipments. An inner join of `vendor` and `shipment` therefore returns five vendors, and the five who have never sent a shipment vanish.
- So the question "which vendors have never sent us anything" cannot be answered by an inner join at all. The rows that would answer it are exactly the ones being dropped.

# LEFT JOIN

LEFT JOIN



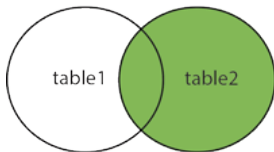
- A LEFT JOIN returns every row from the left table, together with the matching rows from the right table.
- Where the right table has no match, the result still carries the left row, and the right hand columns come back as NULL.

```
SELECT column_name(s)
FROM table1
  LEFT JOIN table2
    ON table1.column_name
       = table2.column_name;
```

- Applied to our ten vendors, this returns all ten rows rather than five, with NULL shipment details against the five who have sent nothing. Combined with IS NULL from Lecture 8, that is how the earlier question gets answered.

# RIGHT JOIN

RIGHT JOIN



- A RIGHT JOIN is the mirror image. It returns every row from the right table, together with the matching rows from the left.

```
SELECT column_name(s)
FROM table1
  RIGHT JOIN table2
    ON table1.column_name
       = table2.column_name;
```

- Any RIGHT JOIN can be rewritten as a LEFT JOIN by swapping the order of the two tables, which is why LEFT JOIN is far more common in practice.
- Note also that MySQL does not support FULL OUTER JOIN, which would keep the unmatched rows from both sides at once. Other database systems do.

Join syntax reference: <https://dev.mysql.com/doc/refman/9.7/en/join.html>

# Modifying Data with SQL

# Introduction

- SQL provides three commands for modifying data in tables, INSERT, UPDATE, and DELETE.
- SQL's data modification statements are rather clumsy for everyday use. Business applications instead provide form-driven data modification, so users do not have to work directly with SQL.
- These statements remain useful while developing and testing a database, such as populating tables with test data.

## Inserting Rows

- There are two variations of INSERT, one to insert a single row, and another to copy one or more rows from another table.

```
INSERT INTO student VALUES (  
    '67676767', 'Nick', 'Mason', 'NM@univ.edu',  
    '666-777-8888', 'Industrial Engineering', 'MS'  
);
```

- The value list must contain one value for every column, in the order the columns were created, and its format must match each column's data type. The DBMS automatically checks any constraints on the table.

```
INSERT INTO student (StudentID, FirstName, LastName,  
    Major, Degree)  
VALUES ('67676767', 'Nick', 'Mason', 'Industrial  
    Engineering', 'MS');
```

- Listing column names explicitly is good practice, even when inserting values for every column, and is especially useful when an INSERT statement is embedded in an application program.

## Copying Existing Rows

```
INSERT INTO StudentNames  
  SELECT FirstName, LastName  
  FROM Student;
```

- The columns in the SELECT must match the columns of the destination table. This example copies rows from Student into a new table, StudentNames.
- Whether data that can be derived from another table should be stored this way depends on the situation. It is worth doing if the derived data is needed frequently and takes a long time to generate.

# Updating Data

- End users typically modify existing data through a form, and the SQL statements to do so are issued by the program behind that form.

```
UPDATE table_name  
SET column1 = new_value, column2 = new_value  
WHERE row_selection_predicate;
```

```
UPDATE ScholarshipRecipient  
SET ScholarshipAmount = 20000, EndDate = '8/31/18'  
WHERE AccountNumber = '1' AND StudentID = '23232323';
```

- Since the predicate here contains a primary key expression, this update affects only one row.

## Updating Multiple Rows

```
UPDATE ScholarshipRecipient  
SET StartDate = '10/1/17'  
WHERE AccountNumber = '3';
```

- This update affects multiple rows, since AccountNumber alone is not a primary key here. If the WHERE clause is omitted entirely, the modification applies to every row in the table.

```
UPDATE ScholarshipFund  
SET AdminFee = TotalAmount * 0.05;
```

- This example assumes a new AdminFee column has already been added to the ScholarshipFund table, and sets it to five percent of the total amount for every row.

# Deleting Rows

- The DELETE statement removes one or more rows, based on the row selection criteria in its WHERE predicate.

```
DELETE FROM table_name  
WHERE row_selection_predicate;
```

```
DELETE FROM ScholarshipRecipient  
WHERE AccountNumber = '5' AND StudentID = '23232323';
```

- This example deletes a single row, representing a student who declined a scholarship.

# DELETE Is Dangerous

```
DELETE FROM ScholarshipRecipient  
WHERE StudentID = '56565656';
```

- This example deletes every scholarship a student held, representing a student who left the university.
- DELETE is a potentially dangerous operation. Omitting the WHERE clause entirely deletes every row in the table.

```
DELETE FROM ScholarshipRecipient;
```

- Deletes also interact with referential integrity, since removing a row still referenced by a foreign key elsewhere can fail or cascade, depending on how that foreign key was declared.

# Referential Integrity Options

- Lecture 8 showed where this choice is made, in the FOREIGN KEY clause of the CREATE TABLE statement, but left the two options unnamed.

```
FOREIGN KEY (foreign_key_columns)
REFERENCES primary_key_table (primary_key_columns)
ON UPDATE update_option
ON DELETE delete_option
```

- Four options are available, and they decide what happens to a child row when its parent is deleted or its key is updated.
  - RESTRICT refuses the operation while any child row still refers to the parent.
  - CASCADE applies the same operation to the children, so deleting a parent deletes them too.
  - SET NULL keeps the children but sets their foreign key to NULL, which requires that the column allows nulls.
  - NO ACTION is checked at the end of the statement, and in MySQL behaves the same as RESTRICT.

# RESTRICT and CASCADE in Practice

- The schema built in the Lecture 8 tutorial declares `ON DELETE RESTRICT`. Deleting a customer who still has orders is therefore refused outright, and the database reports a foreign key constraint failure. Nothing is lost, but nothing happens either.
- Assignment 2 asks for `ON DELETE CASCADE` instead. There, the same delete succeeds, and every order belonging to that customer is removed along with them, even though the delete statement never mentioned the orders table.
- Neither option is the right default. `RESTRICT` protects you from destroying data you did not intend to touch, at the cost of having to clean up the children yourself. `CASCADE` does that cleanup automatically, and will quietly remove a great deal of data if it is pointed at the wrong row.
- The choice is made once, when the table is created, and every later `DELETE` silently obeys it. That is why it is worth deciding deliberately rather than by default.